

ICON structured grid benchmark

CSCS

Introduction

Motivation

- ICON grid is partly structured
- Investigate how expensive are indirect neighbor accesses compared to strided indexes of neighbors
- Investigate optimizations based on strided indexes
- Apply optimizations to current and future `GT4Py` development



ICON grid handling

ICON grid input

- `netcdf` file with neighbor lists
- Use of structured torus grid
 - Generated using [grid-generator](#) from `DKRZ` by `David Strassman`
 - Read using [icon4py_GridManager](#)

Manipulation of ICON grid



Torus grid characteristics

- Torus grid has periodic boundaries
 - Adds complexity to compute strided indexes
 - Not applicable in production scenario of CH

Manipulation of ICON grid



Selection of computation domain

- Filter torus grid to end up with a cartesian grid with 2 level halo
 - 1 would also be sufficient
 - Discard periodic edges
- Calculate **X** and **Y** dimensions based on the distribution of vertices in space
- Order edges and vertices in memory
 - **per-vertex** : **edges** are ordered in memory based on **vertex id**
 - **per-orientation** : **edges** are ordered in memory based on their orientation

Computation domain in red

Manipulation of ICON grid



Selection of computation domain

- Fix `e2v` ordering of edges read by torus grid file to be by default `per-vertex` in [GridManager](#)
 - Some elements in `e2v` didn't follow this ordering by default
- Select between `per-vertex` and `per-orientation` ordering in [GridManager](#)
- Filter halo vertices and edges in [benchmark python script](#)

Computation domain in red

Selected kernels for exploration

- [Nabla4](#)
 - Neighbor tables
 - **e2c2v[4]**
 - **e2ecv[4]**
 - Input fields
 - **u_vert[EdgeDim, KDim]**
 - **v_vert[EdgeDim, KDim]**
 - **primal_normal_vert_v1[EdgeDim * ECVDim]**
 - **primal_normal_vert_v2[EdgeDim * ECVDim]**
 - **z_nabla2_e[OutputEdgeDim, KDim]**
 - **inv_vert_vert_length_gt[OutputEdgeDim]**
 - **inv_primal_edge_length_gt[OutputEdgeDim]**
 - Output field
 - **z_nabla4_e2[OutputEdgeDim, KDim]**

Selected kernels for exploration

- Nabla4 vertical (artificial)
 - Similar to `nabla4` but all input fields are vertical
 - Neighbor tables
 - `e2c2v[4]`
 - `e2ecv[4]`
 - Input fields
 - `u_vert[EdgeDim, KDim]`
 - `v_vert[EdgeDim, KDim]`
 - `primal_normal_vert_v1[EdgeDim * ECVDim, KDim]`
 - `primal_normal_vert_v2[EdgeDim * ECVDim, KDim]`
 - `z_nabla2_e[OutputEdgeDim, KDim]`
 - `inv_vert_vert_length_gt[OutputEdgeDim, KDim]`
 - `inv_primal_edge_length_gt[OutputEdgeDim, KDim]`
 - Output field
 - `z_nabla4_e2[OutputEdgeDim, KDim]`

Selected kernels for exploration

- [Interpolate](#)
 - ■ Neighbor tables
 - **v2e[6]**
 - Input fields
 - **p_e_in[EdgeDim, KDim]**
 - **ptr_coeff_1[OutputVertexDim, 6]**
 - **ptr_coeff_2[OutputVertexDim, 6]**
 - Output fields
 - **p_u_out[OutputVertexDim, KDim]**
 - **p_v_out[OutputVertexDim, KDim]**

Kernel implementations

- Grid structure
 - Unstructured
 - Indirect accesses via neighbor tables
 - Structred
 - Neighbor accesses via strides
- Iteration strategies
 - `gpu_naive`
 - Default iteration strategy in gridtools C++
 - One GPU thread calculates 1 element in horizontal and vertical axis
 - `gpu_kloop`
 - Optional iteration strategy in gridtools C++
 - One GPU thread calculates 1 element in horizontal axis but multiple vertical levels
 - Save neighbors and vertically independent fields in registers and iterate over multiple vertical fields

Kernel implementations

- Kernel combinations
 - Separate
 - Execute `nabla4` kernel and then `interpolate`
 - Inlined
 - Compute `nabla4` output for every input of `interpolate` kernel
 - More computations
 - Less writes to device memory
 - Inlined v2v (unstructured only)
 - Compress `v2e[e2c2v]` neighbor accesses to `v2e2c2v`
 - Read fields for 7 vertices instead of $(6*4=)$ 24 vertices
 - Assumes certain order of vertices in `e2c2v` and edges in `v2e`
 - Inlined_cached
 - Save `nabla4` output in shared memory and then use it in `interpolate` kernel
 - Reduce overcomputations compared to `inlined` implementation

General kernel optimizations

- Occupancy
 - All kernels have been optimized for best occupancy
- `__launch_bounds__` and `__maxnreg__`
 - Launch bounds are applied to all kernels except some that were performing better with register limitation to a certain register number
- Thread Block size
 - Especially for `gpu_naive` implementations, increasing the vertical axis thread block size (`ThreadBlockDim.y/z`) was beneficial since there are more chances to find neighbor tables and vertically independent fields in cache
 - 4-8-9 most used number with 80 vertical levels in total
- `gpu_kloop`
 - For this implementation vertical `GridDim` is set to 1. Iteration number is set based on the `ThreadBlockDim.y/z` and `KDim`. Exception are the `inline_cached` versions where the shared memory size is limiting the number of iterations possible
 - Optimized number of iterations per kernel. 8-40 iterations. 8-20 usually have the best performance

Notes for specific kernels

Results

- Nabla4
- Interpolate
- Nabla4 & interpolate
- Nabla4_vertical & interpolate

Conclusions

Next steps

Questions?

Resources

- [GitHub Repository](#)